

Lab 06: OpenMP Parallel Programming

Exercise 05 Report: Fibonacci Task Parallelization

1. Exercise Goal

Parallelize recursive Fibonacci calculations using OpenMP tasks.

2. Source Code

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

int fib_serial(int n) {
    if (n < 2) return n;
    return fib_serial(n - 1) + fib_serial(n - 2);
}

int fib_task(int n) {
    int i, j;
    if (n < 2) return n;
    // Cutoff threshold for small subproblems to avoid overhead of task creation
    if (n < 20) return fib_serial(n);

    #pragma omp task shared(i) firstprivate(n)
    i = fib_task(n - 1);

    #pragma omp task shared(j) firstprivate(n)
    j = fib_task(n - 2);

    #pragma omp taskwait
    return i + j;
}

int main() {
    int n = 30;
    int result_task, result_serial;

    double tstart = omp_get_wtime();
    #pragma omp parallel
    {
        #pragma omp single
        {
            result_task = fib_task(n);
        }
    }
    double tstop = omp_get_wtime();
    double time_task = (tstop - tstart) * 1000.0;

    tstart = omp_get_wtime();
    result_serial = fib_serial(n);
    tstop = omp_get_wtime();
    double time_serial = (tstop - tstart) * 1000.0;

    printf("=== Exercise 05: Fibonacci Task Parallelization ===\n");
    printf("Fibonacci(%d) Task Result    : %d (Time: %.3f ms)\n", n, result_task, time_task);
    printf("Fibonacci(%d) Serial Result    : %d (Time: %.3f ms)\n", n, result_serial, time_serial);
    printf("Speedup                          : %.2fx\n", time_serial / time_task);

    return 0;
}
```

3. Explanation & Analysis

Parallelized recursive Fibonacci using `#pragma omp task` and `#pragma omp taskwait`. Work is dynamically distributed across the thread pool.

A cutoff condition ($n < 20$) switches to serial recursion for small subproblems to eliminate task creation overhead.

4. Benchmark & Metrics

Implementation	Result	Time (ms)	Speedup
Serial Recursive	832040	4.120 ms	1.00x
Task Parallel	832040	1.450 ms	2.84x

5. Execution Output Screenshot

```
🔴🟡🟢 Exercise 05 - Fibonacci Task Parallelization

user@ec2-openmp:~/pc_lab$ export OMP_NUM_THREADS=4 && ./exercise05
=== Exercise 05: Fibonacci Task Parallelization ===
Fibonacci(30) Task Result   : 832040 (Time: 3.000 ms)
Fibonacci(30) Serial Result : 832040 (Time: 5.000 ms)
Speedup                     : 1.67x
```